

Delegating Computation with Universal Setup

A. Author, B. Author, C. Author

Institute of Cryptography, Example University

Abstract.

We study the problem of delegating computation to untrusted servers while keeping both privacy and verifiability. Prior work shows that a structured reference string can be constructed in a universal and updatable fashion, which removes one of the main obstacles in deployment. Our construction follows the permutation argument of Bayer and Groth, but evaluates the argument over a multiplicative subgroup rather than over coefficients of monomials. As a result, both the permutation argument and the arithmetization step become substantially simpler, and the prover running time drops accordingly. We measure the end-to-end cost on circuits of increasing size and report the group exponentiations required by each phase. The verifier run time remains polylogarithmic in the circuit size, while the proof length stays logarithmic. We also discuss the preprocessing phase and the generation of the structured reference string.

1 Introduction

A related convenience is that multiplicative subgroups interact well with Lagrange bases. Suppose H is a multiplicative subgroup of order n and x lies in H . The polynomial that vanishes on the rest of the subgroup and takes value one at x has a very sparse representation. To see why this matters, consider the cost of checking polynomial identities: with a sparse representation the check reduces to a handful of group operations instead of a full interpolation. We then combine this with the grand product argument and show that the resulting protocol is complete and knowledge sound under the discrete logarithm assumption in the random oracle model.

$$\lambda_{\xi}(\Xi) = \chi_{\xi}(\Xi \perp v - 1) / (\Xi - \xi)$$

2 Efficiency Analysis

We study the problem of delegating computation to untrusted servers while keeping both privacy and verifiability. Prior work shows that a structured reference string can be constructed in a universal and updatable fashion, which removes one of the main obstacles in deployment. Our construction follows the permutation argument of Bayer and Groth, but evaluates the argument over a multiplicative subgroup rather than over coefficients of monomials. As a result, both the permutation argument and the arithmetization step become substantially simpler, and the prover running time drops accordingly. We measure the end-to-end cost on circuits of increasing size and report the group exponentiations required by each phase. The verifier run time remains polylogarithmic in the circuit size, while the proof length stays logarithmic. We also discuss the preprocessing phase and the generation of the structured reference string.

Scheme	Prover (G1 exp)	Proof size	Succinct
Groth16	$3n + m$	3 G1	yes
Sonic	$18n$	4 G1	no
This work	$2n$	2 G1	yes

References

- [1] S. Bayer and J. Groth. Efficient zero-knowledge argument for correctness of a shuffle, 2012.
- [2] J. Groth. On the size of pairing-based non-interactive arguments, 2016.
- [3] M. Maller, S. Bowe, M. Kohlweiss and S. Meiklejohn. Sonic, 2019.